

Visualization of Lattice-Based Cryptanalysis Algorithms

Muhammad Imam Chaidar Mujaddid Al-Ghiffari¹[0000-1111-2222-3333] and
Second Author^{2,3}[1111-2222-3333-4444]

Fakultas Ilmu Komputer Universitas Indonesia, Jl. Prof. DR. Sudjono D.
Puspongoro, Pondok Cina, Kecamatan Beji, Kota Depok, Jawa Barat 16425
<https://cs.ui.ac.id/>

Abstract. The abstract should briefly summarize the contents of the paper in 150-250 words.

Keywords: Cryptography · Cryptanalysis · Post-Quantum · Lattice · LLL · BKZ.

1 Background

Cryptanalysis is the technique of analysing and breaking a ciphertext to obtain the plaintext (A. Al-Sabaawi, 2020). The history and the development of cryptanalysis itself can't be separated from the history and development of cryptography. As the cryptography techniques evolve, the cryptanalysis techniques also evolve in order to break the cryptography techniques.

One of the earliest known recorded cryptanalysis technique is the frequency analysis technique, which was used by Arab mathematician Al-Kindi in the 9th century to break monoalphabetic substitution ciphers. This technique relies on the fact that in any given language, certain letters and combinations of letters occur with varying frequencies. By analysing the frequency of letters in the ciphertext, a cryptanalyst can make deduce the likely substitutions used in the cipher.

The frequency analysis technique was arguably the most common and efficient cryptanalysis technique used for breaking classic ciphers. However, as the cipher techniques became more complex, this cryptanalysis technique became less effective. Overall, during and after World War II, mathematics became more and more crucial in both cryptography and cryptanalysis. And the modern cryptanalysis techniques are mostly based on complex mathematical concepts and algorithms, like number theory, algebra, and probability theory.

Since the most common form of modern cryptography is key-based cryptography (symmetric and asymmetric), with the onset of the development of quantum computing and subsequently quantum cryptanalysis, many commonly used modern cryptographic algorithms are threatened by quantum cryptanalysis. For example, Shor's algorithm - a quantum algorithm to finding prime factors of an integer, can efficiently factor large integers and compute discrete

logarithms. Which endangers the security of commonly used asymmetric cryptographic algorithms like RSA and ECC. Not only asymmetric cryptographic algorithms are threatened by this, Grover's algorithm - also known as Quantum Search Algorithm, can search unsorted databases quadratically faster than classical algorithms, which threatens the security of symmetric-key cryptosystems by effectively halving their key lengths.

Which is why cryptographers around the world already started to develop cryptographic algorithms that can resist quantum cryptanalysis. These newer and stronger cryptographic algorithms are called post-quantum cryptography (PQC). Examples of PQC algorithms include Kyber and Dilithium, which are based on the hardness of lattice problems. In fact, lattice-based cryptographic algorithms dominate the PQC. The lattice-based PQC algorithms are indeed able to resist quantum computer-based cryptanalysis. For example, the quantum cryptanalysis technique of Shor's Algorithm can't break it efficiently.

However, just like how the historical pattern of cryptanalysis and cryptography goes in our history, as cryptographers developed lattice-based cryptographic algorithms, the cryptanalysis experts also developed lattice-based cryptanalysis techniques in order to break the lattice-based cryptographic algorithms. The most common lattice-based cryptanalysis technique is the LLL algorithm, which is a polynomial-time algorithm for finding a short and nearly orthogonal basis for a lattice. This algorithm can be used to solve the shortest vector problem (SVP) and the closest vector problem (CVP), which are both important issues in lattice-based cryptography. Apparently, for now, the most promising lattice-based cryptanalysis technique is the BKZ algorithm, which is basically the upscaled version of the LLL algorithm. Compared to LLL, BKZ can find even shorter vectors in a lattice.

The problem with these lattice-based cryptanalysis techniques is that in general, they are very complex and hard to understand, especially for people who are new to the field of cryptography and cryptanalysis. This is where the visualization of these algorithms can be very helpful. However, most visualizations of these algorithms don't visualize the underlying mathematical concepts in an intuitive way. By visualizing these algorithms in an intuitive way, we can make them easier to understand and more accessible to a wider audience.

2 Related Works

2.1 lll-attack-runner

lll-attack-runner is a Javascript-based interactive web application for running advanced lattice basis reduction algorithms such as LLL and BKZ. It is developed by Zachary Robert Bennett with the latest update was in January 2026. Because we aim to create a Javascript-based interactive web application for visualizing lattice-based cryptographic and cryptanalysis algorithms, in which two of them are LLL and BKZ, we will be using lll-attack-runner as the backend for our visualization of the LLL and BKZ algorithms. However, since we want to

visualize the algorithms in a comprehensive and intuitive way, we will develop our own frontend for the visualization.

2.2 fpylll

fpylll is the Python wrapper of fplll. fplll itself is a solver tool for LLL and BKZ cryptanalysis algorithms, and was developed by fplll development team. fpylll itself was maintained by Martin Albrecht. Like fplll, fpylll lacks visualization tools for the LLL and BKZ algorithms.

2.3 Herramienta de software para la visualización de la criptografía basada en retículas

Authored by Jaziel D. Flores Rodríguez and his fellow colleagues, this research paper was aimed to create a visualization tool for post-quantum cryptographic algorithms. The main focus was to graphically visualize mathematical concepts behind these post-quantum cryptographic algorithms, particularly lattice concepts which are the basis of these algorithms. The tool which was developed in this research, successfully visualizes these lattice-based post-quantum cryptographic algorithms within dimensions of 2, 3, 4, and 5. However, the LLL process for dimensions higher than 3 is still not implemented in this tool, and the BKZ algorithm is also not implemented in this tool. Which is why we will be implementing the LLL process for dimensions higher than 3 in our visualization tool, as well as implementing the BKZ algorithm which is not implemented in this tool.

3 Architecture

3.1 Backend

We will use Zachary Robert Bennett's lll-attack-runner, which is an interactive and Javascript-based lattice-based cryptanalysis solver, as the backend for our visualization of the LLL and BKZ algorithms. We only use three source codes from lll-attack-runner, which are : lll.ts, bkz.ts, and types.ts. The lll.ts, per its filename, does the LLL algorithm process, and the same goes for bkz.ts. Both source codes needs LLLStep interface from types.ts, and for bkz.ts, it also needs LLLRun from lll.ts. Which makes sense, consider the usage of LLL algorithm in BKZ algorithm.

In each of our algorithm modules, each algorithm has its own folder, which in each of these folders, there is a single source code written in Typescript, with naming convention of the algorithm name written in lowercase. Each of these source codes are responsible for running the algorithm process which corresponds to their name. Within each of these source codes, we will use imported functions from Zachary Robert Bennett's lll-attack-runner source codes to run the algorithm process.

3.2 Frontend

For the frontend part, we will use Next.js, which is a React-based framework for building web applications. The usage of Next.js is intended to create a user-friendly interface for our visualization of the LLL and BKZ algorithms. The frontend will allow users to input their own lattice-based cryptanalysis problems and visualize the steps of the LLL and BKZ algorithms in an intuitive way.

For the visualization part, we will use D3 for the visualization part, combined with React to create matrices and vectors visualizations. Our secondary objective for the frontend is to create a visualization which have animations for the steps of these algorithms. We believe the animated version of these visualizations can be understand easier by the users.

3.3 A Subsection Sample

Please note that the first paragraph of a section or subsection is not indented. The first paragraph that follows a table, figure, equation etc. does not need an indent, either.

Subsequent paragraphs, however, are indented.

Sample Heading (Third Level) Only two levels of headings should be numbered. Lower level headings remain unnumbered; they are formatted as run-in headings.

Sample Heading (Fourth Level) The contribution should contain no more than four levels of headings. Table 1 gives a summary of all heading levels.

Table 1. Table captions should be placed above the tables.

Heading level	Example	Font size and style
Title (centered)	Lecture Notes	14 point, bold
1st-level heading	1 Introduction	12 point, bold
2nd-level heading	2.1 Printing Area	10 point, bold
3rd-level heading	Run-in Heading in Bold. Text follows	10 point, bold
4th-level heading	<i>Lowest Level Heading.</i> Text follows	10 point, italic

Displayed equations are centered and set on a separate line.

$$x + y = z \tag{1}$$

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead (see Fig. 1).

Theorem 1. *This is a sample theorem. The run-in heading is set in bold, while the following text appears in italics. Definitions, lemmas, propositions, and corollaries are styled the same way.*

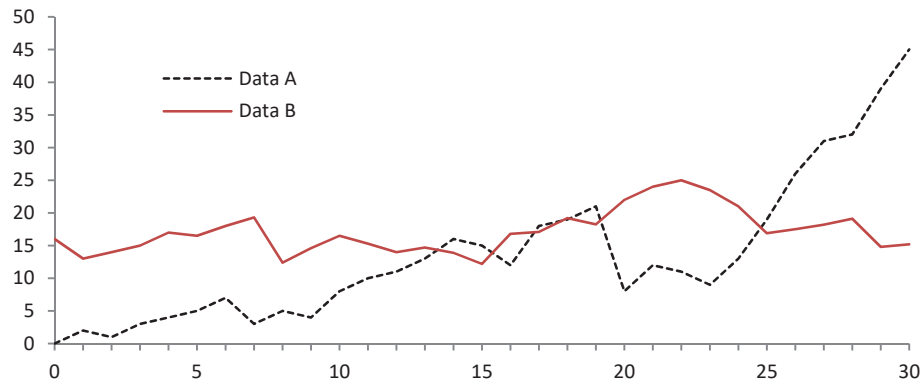


Fig. 1. A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.

Proof. Proofs, examples, and remarks have the initial word in italics, while the following text appears in normal font.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable. The following bibliography provides a sample reference list with entries for journal articles [1], an LNCS chapter [2], a book [3], proceedings without editors [4], and a homepage [5]. Multiple citations are grouped [1–3], [1, 3–5].

Acknowledgments. A bold run-in heading in small font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

Disclosure of Interests. It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a bold run-in heading in small font size beneath the (optional) acknowledgments¹, for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

References

1. Author, F.: Article title. *Journal* **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) *CONFERENCE 2016, LNCS*, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). <https://doi.org/10.1007/1234567890>

¹ If EquinOCS, our proceedings submission system, is used, then the disclaimer can be provided directly in the system.

3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25